

Figure 1 -- Expression grammar (S-expression syntax)

Form	Meaning
<code>n</code>	integer literal, e.g. 42, -3
<code>true false</code>	boolean literal
<code>x</code>	variable reference
<code>(fun x e)</code>	lambda; curry multi-arg via nested <code>(fun x (fun y e))</code>
<code>(e1 e2)</code>	application
<code>(let x e1 e2)</code>	let <code>x = e1</code> in <code>e2</code>
<code>(if e1 e2 e3)</code>	conditional
<code>(+ e1 e2) (- e1 e2) (* e1 e2)</code>	<code>Int -> Int -> Int</code>
<code>(< e1 e2) (== e1 e2)</code>	<code>Int -> Int -> Bool</code>
<code>(pair e1 e2)</code>	pair construction
<code>(fst e) (snd e)</code>	pair projection
<code>nil</code>	empty list, polymorphic in element type
<code>(cons e1 e2)</code>	list construction
<code>(isnil e) (head e) (tail e)</code>	list operations

Figure 1 -- Expression grammar.

Figure 2 -- The type system

Int, Bool

base types

T1 -> T2

function type

T1 * T2

pair type

List T

homogeneous list type

'a, 'b, 'c, ...

type variables (inference-internal)

Figure 2 -- The type system.

Figure 3 -- This packet's generalization rule (stricter than standard ML's value restriction)

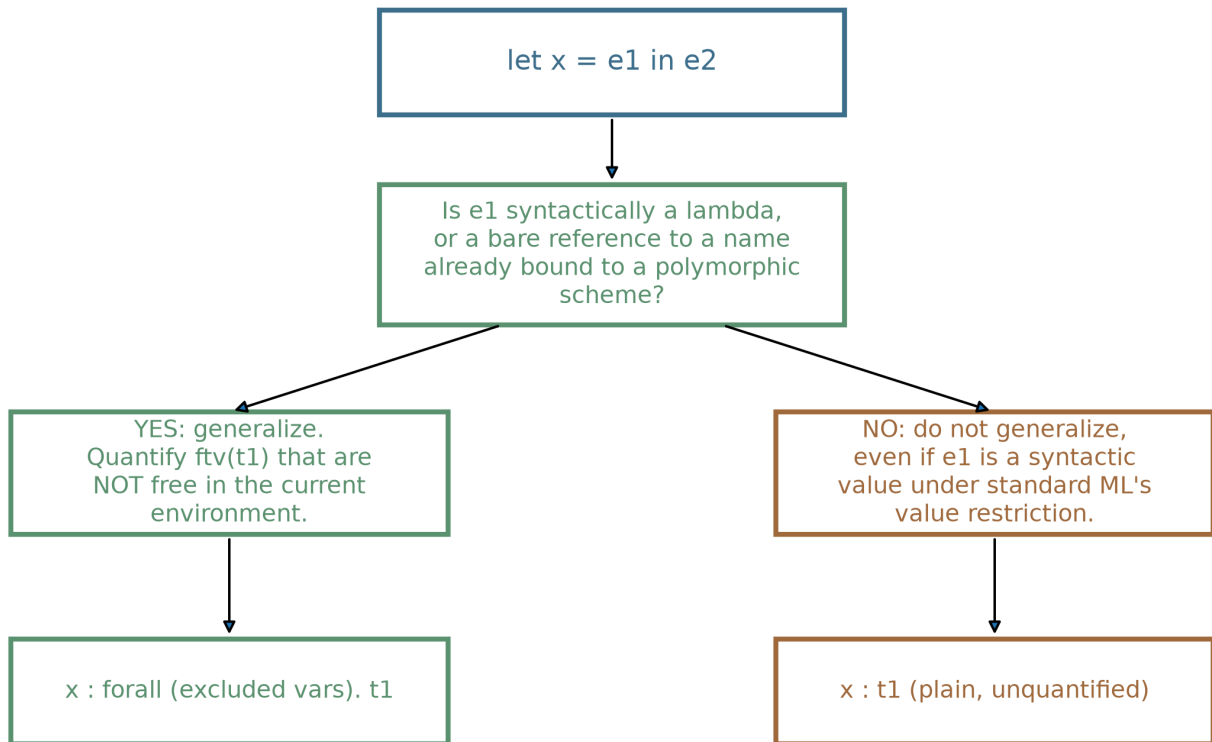


Figure 3 -- This packet's generalization rule.

Figure 4 -- The fixed 27-binding program (this task's entire input)

```
(program
  (def five 5)
  (def mk1 (fun x (let keep (fun z x) (pair (keep true) (keep five)))))
  (def r1 (fst (mk1 7)))
  (def mk2 (fun x (let keep (fun z x) (pair (keep r1) (keep false)))))
  (def r2 (snd (mk2 9)))
  (def mk3 (fun x (let keep (fun z x) (cons (keep r2) (cons (keep 2) nil)))))
  (def r3 (head (mk3 11)))
  (def mk4 (fun x (let keep (fun z x) (pair (keep r3) (keep 0)))))
  (def r4 (fst (mk4 13)))
  (def mk5 (fun x (let keep (fun z x) (pair (keep r4) (keep r1)))))
  (def r5 (fst (mk5 15)))
  (def mk6 (fun x (let keep (fun z x) (cons (keep r5) (cons (keep r2) nil)))))
  (def r6 (head (mk6 17)))
  (def mk7 (fun x (let keep (fun z x) (pair (keep (< r6 0)) (keep r3)))))
  (def r7 (snd (mk7 19)))
  (def mk8 (fun x (let keep (fun z x) (pair (keep r7) (keep (+ r4 r7)))))
  (def r8 (fst (mk8 21)))
  (def mk9 (fun x (let keep (fun z x) (cons (keep r8) (cons (keep r5) nil)))))
  (def r9 (head (mk9 23)))
  (def mk10 (fun x (let keep (fun z x) (pair (keep r9) (keep (== r8 r9)))))
  (def r10 (fst (mk10 25)))
  (def wrap_in_list (fun x (cons x nil)))
  (def list_of_r10 (wrap_in_list r10))
  (def id (fun x x))
  (def use_id_on_r10 (id r10))
  (def alias_id id)
  (def use_alias_id_on_pair (alias_id (pair r10 five)))
)
```

Figure 4 -- The fixed 27-binding program (this task's input).

1. THE LANGUAGE (Figures 1-2)

A small, pure (no effects, no mutation) functional language in S-expression syntax. Recover the full grammar from Figure 1 and the type system from Figure 2: Int, Bool, function types, pair types, list types, and type variables used internally during inference.

2. THE GENERALIZATION RULE (Figure 3)

At every let binding, this packet's own rule decides whether the bound name gets a polymorphic (quantified) type scheme or a plain, unquantified one -- and it is stricter than standard ML's value restriction, not safe to assume from memory. Generalize only when the bound expression is syntactically a lambda, or a bare reference to a name already bound to a polymorphic scheme. For every other form -- integer or boolean literals, applications, if-expressions, pair or list constructions or projections, anything else -- do not generalize, even though standard ML's value restriction would allow it for several of these forms: bind the name to its plain type instead. When generalizing, quantify exactly the type variables free in the bound expression's type that are not free in the current typing environment; quantifying one that is still free in the environment is unsound and is this packet's central correctness trap, since the resulting variable is actually tied to an outer, not-yet-resolved lambda parameter, not independently polymorphic.

3. UNIFICATION

Standard syntax-directed unification with an occurs-check: unifying a type variable with a type that contains that same variable must fail (it would require an infinite type), never silently succeed.

4. THE FIXED PROGRAM (Figure 4)

The entire input is the 27-binding program in Figure 4. It is a real dependency chain, not 27 independent facts: most bindings call or reference 2-4 earlier ones, often through a "capturing factory" pattern (a lambda containing a nested let that closes over the outer, not yet generalized parameter) -- getting the generalization rule wrong on one binding corrupts the substitution threaded through everything built on top of it. Every binding's final type is fully determined (ground or validly polymorphic) by this program -- there is no ambiguity to resolve beyond correctly applying the stated rules.

5. INDEPENDENT VERIFICATION

Build a separately-coded bidirectional type checker -- not a second inferencer running the same algorithm, but one that verifies each binding's claimed type by checking its body against that claim, top-down -- sharing only immutable input constants (the grammar, the program text) with the primary implementation, never importing its substitution, environment, or any computed type. Run two required adversarial mutations against the checker: a self-application expression ($\text{fun } x \rightarrow (x \ x)$), which requires an infinite type and must be rejected; and a claimed type scheme that over-generalizes (quantifies a type variable that is actually still free in its own enclosing environment), which must also be rejected even though it may look like a plausible, internally-consistent type at first glance. The checker must accept the true program's own correct types while rejecting both.

6. CERTIFICATION

Build the serialization as exactly these lines, in this exact order, each line `BindingName=Type`, with no spaces, built from the type's printed form: `Int`, `Bool`, `(T1->T2)`, `(T1*T2)`, `List(T)`, each function and pair former fully parenthesized with no precedence elision. For a generalized binding whose scheme has at least one quantified variable, prefix with "forall ", then the quantified variables renamed to `t0`, `t1`, `t2`, ... in first-occurrence order reading the type left to right, then " . ", then the type body using those names; a binding that is generalization-eligible but ends up with zero free type variables is printed as a plain type, with no forall prefix. List the 27 binding lines in the SOURCE ORDER they appear in Figure 4, not alphabetical, not dependency order. Join the literal header line "TYPECHAIN9-CERT-V1" and the 27 binding lines with a single newline character, UTF-8 encode, hash with SHA-256; report the first 16 hex characters as the trace-integrity certificate.

Deliver: inference engine source; independent checker source; the full typing output for all 27 bindings; a findings report; certification evidence (both adversarial mutation rejection results plus the trace hash); and an engineering memo explaining the causal chain from the generalization rule through the program's actual dependency structure to the final claimed types. Base every reported type on your own executed implementation -- never hand-derive or embed a precomputed type or hash as a substitute for running the delivered engine.